

Fisingas

A Self-Hosted Phishing-Recognition Training and
Assessment Platform Built on Django, React and Docker

Tomas Vanagas

`tomas.vanagas@knf.vu.lt`

August 2026

Abstract

Fisingas is a self-hosted web platform for teaching and assessing phishing-email recognition, built for classroom use at Vilnius University Kaunas Faculty. Students register themselves, receive a system-generated access code, and take a test in which they examine screenshots of real-looking emails — complete with hoverable link previews simulated over the image — and judge each one as genuine or phishing, answering follow-up questions about the tell-tale signs. Administrators manage the question bank, watch student progress live, and project an auto-refreshing leaderboard interleaved with presentation slides. The system is packaged as a single Docker Compose stack of nine containers behind one Caddy reverse proxy: a React single-page application, a deliberately thin Django JSON API, a PostgreSQL database, and a set of gateway-authenticated operator tools (database UI, file managers, API documentation). The central design concern is *grade integrity on mutable content*: dealing a test freezes a denormalized snapshot of every question into the student’s answer rows, so editing or deleting questions can never change grades already given. This paper describes the architecture, the data model and its snapshot discipline, the authentication scheme, the test lifecycle and the exact grading arithmetic, the web interface for both roles, and the procedures for deploying and operating the service. All statements in this paper are derived directly from the source code of the system.

Contents

1	Introduction	3
2	System Architecture	3
2.1	Services	3
2.2	Routing	4
2.3	Security posture	4
3	Data Model	5
3.1	Entities	5
3.2	Snapshot on deal	5
3.3	The image store	6
4	Authentication and Access Control	6
4.1	Accounts and roles	6
4.2	Sessions	7
4.3	Per-view authorization	7
4.4	Gateway-level authorization	7

5	The Test Lifecycle	7
5.1	One student, end to end	7
5.2	Grading	9
6	The Web Interface	9
6.1	Technology	9
6.2	The student experience	10
6.3	The administrator experience	10
6.4	Public displays	11
7	Deployment and Operations	11
7.1	Prerequisites	11
7.2	Starting the stack	11
7.3	Data retention	12
7.4	Configuration reference	12
7.5	Development mode	12
7.6	Regression tests	13
8	Limitations and Future Work	13
9	Conclusion	14

1 Introduction

Phishing remains the most common initial access vector in practice, and the most effective countermeasure available to an educator is rehearsal: show people convincing forgeries next to genuine messages and let them practise telling the two apart. Running such an exercise for a lecture hall of students raises requirements that generic quiz software meets poorly:

- The material is *visual*. A phishing exercise lives on pixel-accurate screenshots of email clients, and the crucial skill — hovering a link to inspect its real destination — must work on a static image.
- Enrolment must be instant and anonymous-friendly. Students register themselves with a nickname during the lecture and receive a machine-generated access code; no email accounts, no roster imports.
- The exercise is *live*. The lecturer needs a real-time view of who is still answering, and a projected leaderboard turns assessment into a game.
- Content changes constantly — questions are added, reworded and retired between (and during) course editions — yet a grade, once given, must remain exactly reproducible.
- The platform must run self-hosted on a single university server, operable by one person with Docker.

Fisingas addresses these requirements with deliberately boring technology: a Django API that “only moves data around”, PostgreSQL for every byte of state (including the screenshot images themselves), a React single-page application, and Docker Compose for orchestration. There is no message queue, no cache layer, no cloud dependency and no JavaScript build step at runtime. The one place where the design is opinionated is the data model: the test a student receives is a *frozen copy* of the questions, not a reference to them (Section 3).

The remainder of this paper is organised as follows. Section 2 presents the container topology, routing and security posture. Section 3 describes the data model and the snapshot discipline. Section 4 specifies the authentication and authorization scheme. Section 5 follows one student through the test and gives the grading arithmetic. Section 6 describes the web interface for students, administrators and the projector. Section 7 is an operator’s guide: deployment, auxiliary services, data retention and the configuration reference. Section 8 discusses known limitations.

2 System Architecture

2.1 Services

The stack consists of nine containers declared in a single `docker-compose.yml`. They are summarised in Table 1 and depicted in Figure 1.

The Django backend is intentionally thin: plain function views with no Django REST Framework, no `django.contrib.admin` and no `django.contrib.auth` — the installed-apps list contains only the session framework and the three project apps (`users`, `phishing_test`, `leaderboard`). All content management happens through the React admin interface; all database access goes through the ORM — the only raw SQL in the codebase is the migration that converted the timestamp columns to native datetimes (Section 3). Every request runs in a transaction (`ATOMIC_REQUESTS = True`), so a half-written test state cannot persist.

Container	Technology	Role
fisingas-endpoint	Caddy 2.11	Sole entry point; routing, security headers,
fisingas-vite	React 19 + Vite 7, served by Caddy	Web frontend (SPA)
fisingas-django	Django 5.2 + Gunicorn (Python 3.13)	JSON API: auth, test flow, grading
fisingas-postgres	PostgreSQL 17	All persistent state, including images
fisingas-cron	BusyBox crond + Docker CLI	Nightly data-retention sweep
fisingas-filebrowser-dropbox	Filebrowser (no-auth build)	Shared file drop for staff
fisingas-filebrowser-slides	Filebrowser (no-auth build)	Upload of projector slides
fisingas-dbgate	DBGate 7.2	Database management UI
fisingas-swagger	Swagger UI	Interactive API documentation

Table 1: Services in the Docker Compose stack.

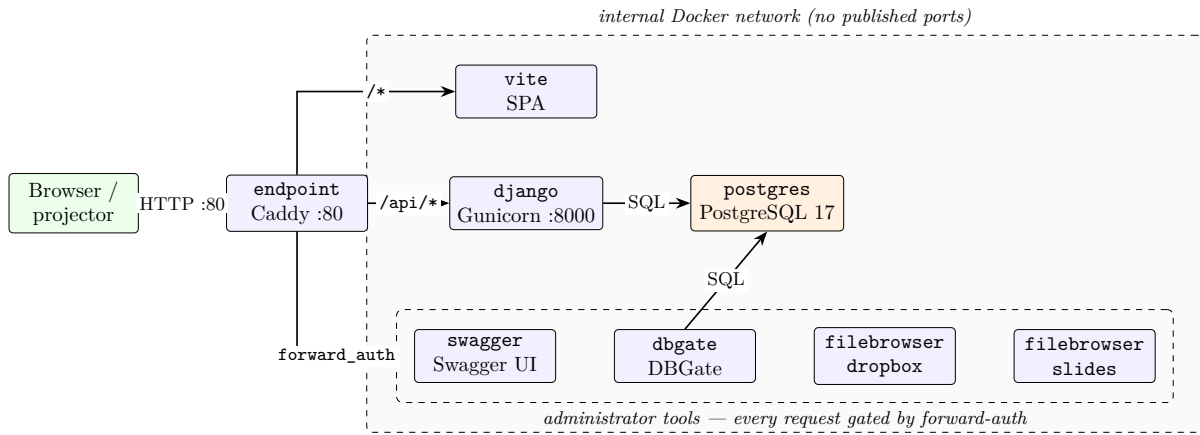


Figure 1: Service topology. Only the Caddy endpoint publishes a host port; every other container is reachable solely over the internal Docker network. The two filebrowser instances serve the bind-mounted `_DATA/dropbox` and `_DATA/slides` directories; Django reads the same slides directory for the projector mode. The cron container is not shown: it runs with `network_mode: none` and reaches Django only through the Docker socket (`docker exec`).

2.2 Routing

The endpoint Caddy instance listens on port 80 and routes by path, in order:

- `/filebrowser/dropbox/*`, `/filebrowser/slides/*`, `/dbgate/*` and `/swagger*` are reverse-proxied to the respective tool, but only after a `forward_auth` subrequest to `fisingas-django:8000 /api/checkauth/admin` confirms the caller holds an administrator session (Section 4.4);
- `/api/*` is proxied to the Django backend with no gate at the proxy — Django decides per endpoint who may call what;
- everything else falls through to the frontend container, where a second Caddy instance serves the pre-built Vite bundle as static files with an SPA fallback (`try_files {path} /index.html`), so deep links such as `/admin/students/42` resolve client-side.

2.3 Security posture

Hardening measures applied uniformly across the stack:

- **Single ingress.** Only the endpoint container publishes a port. Django, PostgreSQL and all tools are unreachable from the host network.
- **Same-origin enforcement at the edge.** Caddy rejects with 403 any request whose

`Origin` header is present and does not match the request host. This is the stack’s CSRF defence — Django’s CSRF middleware is deliberately not installed (Section 8 discusses the boundary of this choice).

- **Content-Security-Policy.** A strict CSP (`default-src 'self'`; images additionally from `data:` and `blob:`, which the slide projector needs) plus `frame-ancestors 'self'` and a `Strict-Transport-Security` header with a one-year max-age.
- **Read-only filesystems and non-root execution.** The endpoint, frontend, backend and database containers run with `read_only: true` as user `1000:1000`; writable paths are limited to explicit volumes and `tmpfs` mounts (Django gets a `tmpfs` at `/tmp` for upload spooling).
- **Secrets out of the compose file.** The Django signing key and the DBGate credentials are generated by the deploy script into `.env` (Section 7.2); neither `.env` nor the deployed `docker-compose.yml` is tracked in git.

3 Data Model

3.1 Entities

All state lives in PostgreSQL in eleven tables (Table 2). Every timestamp is a native `timestampz` column holding UTC; the API publishes it as ISO-8601 in local time with the explicit offset (`2026-08-26T19:09:07+03:00`, `+02:00` in winter) and `null` for “never”. All comparisons — the dashboard’s “active in the last 30 minutes” window, the retention sweep, the frontend’s date filters — are made on real datetimes, so the daylight-saving transitions are harmless and the display is correct in any browser timezone.

Table	Purpose
<code>users_systemuser</code>	Administrator accounts (email, bcrypt hash, enabled flag)
<code>users_student</code>	Student accounts (username, plaintext access code, finished flag)
<code>users_setting</code>	Key–value settings; the only key used is <code>PhishingTestSize</code>
<code>phishing_test_questionimage</code>	Email screenshots, stored as binary blobs; upload-only
<code>phishing_test_question</code>	The live, editable question bank
<code>phishing_test_questionoption</code>	Follow-up checkbox options of a question
<code>phishing_test_questionlink</code>	Clickable link areas on an image (fractional coordinates)
<code>phishing_test_answer</code>	<i>Frozen</i> per-question snapshot + the student’s verdict
<code>phishing_test_answerselectedoption</code>	<i>Frozen</i> per-option snapshot + the student’s checkbox
<code>phishing_test_testresult</code>	<i>Frozen</i> whole-test totals, written once at finish
<code>django_session</code>	Server-side session storage

Table 2: Database tables. “Frozen” tables are written when a test is dealt or finished and are never affected by later edits to the bank.

3.2 Snapshot on deal

The core invariant of the system is that *a grade must be reproducible forever, no matter what happens to the question bank*. This is achieved by denormalization at deal time: when a student first requests their questions, the system copies — into `Answer` and `AnswerSelectedOption` rows — each dealt question’s text, its correct verdict, a link to its image, and every option’s text and expected value. The copies deliberately reference the question by a plain integer, not a foreign key, so deleting the question later cannot cascade into a dealt test. Grading (Section 5.2) reads exclusively from these frozen rows.

Two uniqueness constraints — `(student, question_id)` on answers and `(student, question_id, option_id)` on options — guarantee that a student can never hold two snapshots of one question. Concurrency itself is handled one level up: every request on the test endpoints first updates the student’s “last seen” timestamp, which takes that student’s row lock in PostgreSQL, so two simultaneous first requests are serialised — the second waits, then sees the first one’s rows and skips dealing. Should the constraint fire regardless, the request logs it and serves the snapshot that landed; any other integrity failure is logged and re-raised rather than answered with an empty test. Combined with per-request transactions, a partially dealt test cannot exist.

When the student finishes, a third freeze happens: the computed totals are written once into `TestResult`, under the same row lock, so an autosave that is still committing is included in the frozen totals — the grade the lists show and the per-question review can never disagree. From then on every list view (admin table, public leaderboard) serves that student from a single stored row instead of re-judging their answers, so the hot pages do constant work per finished student and live judging is reserved for students still answering.

3.3 The image store

Screenshots are the only binary payload and get their own upload-only table. A new question is created *by* uploading an image (Section 6.3); the file — extension-checked against `png|jpg|jpeg|gif` and capped at 5 MB — is stored as raw bytes in PostgreSQL, and a wrapper question row is created around it. When serving, the content type is sniffed from the magic bytes of the blob rather than trusted from the upload.

Both the live question and the frozen answer snapshots reference the image with `PROTECT`, and no code path deletes images: deleting a question keeps its screenshot, and `GET /api/phishingpictures/{id}` falls back to the snapshot reference when the question no longer exists, so a graded test can always re-display its emails.

The clickable link areas (`QuestionLink`) hang off the *image*, not the question — their coordinates (stored as fractional text values) only make sense on that exact bitmap, and this placement means old graded tests keep their hoverable links even after the question is deleted.

4 Authentication and Access Control

4.1 Accounts and roles

There are exactly two kinds of account, held in separate tables, and the login name itself encodes the role: administrators sign in with an email address, students with a generated username. The login endpoint distinguishes them by whether the submitted name contains `@` — safe because student usernames are normalised at registration to the alphabet `A-Z 0-9 _` (uppercased, all other characters stripped), so the two namespaces cannot collide.

Administrator passwords are hashed with `bcrypt` (12 rounds), with an 8-character minimum and a 72-byte maximum — `bcrypt`’s input limit; a longer password would silently authenticate on its truncated prefix — enforced when accounts are created or updated, and an email address must be unique on edit as well as on create. Student credentials are system-generated 8-digit access codes (`10000000–99999999`) stored and compared *in plaintext* — a documented design decision: the code is a system-issued access token shown back to the student on demand, not a user-chosen secret that could be reused elsewhere. A failed login for an unknown name still executes a `bcrypt` comparison against a dummy hash, so response timing does not reveal whether an account exists.

4.2 Sessions

Django’s database-backed session framework is the only piece of `django.contrib` in use. Logging in stores a single value — the login name — in a new session row and sets an `HttpOnly` cookie named `session`, configured to expire when the browser closes. Logging out is server-side: `POST /api/logout` deletes the session row and clears the cookie, so a cookie value captured earlier (shared or kiosk machines) is dead from that moment; the login page issues that call whenever it is opened, which is how the “Atsijungti” link works. On every request the session name is re-resolved against the account tables, and disabled accounts are treated as non-existent — a deactivated administrator or student loses any live session on their next request.

4.3 Per-view authorization

A small custom decorator replaces `django.contrib.auth: @login_required` rejects anonymous requests with 401 and attaches the resolved account to the request; role checks are then per-view. Three access tiers result: public endpoints (login, logout, registration, leaderboard, slides), authenticated endpoints (question images and their link areas), and admin endpoints — two of which (`student detail` and `student answers`) also accept the student reading *their own* record, which is how the results page works.

4.4 Gateway-level authorization

The operator tools (DBGate, both filebrowsers, Swagger UI) have no authentication of their own. Instead, Caddy performs a `forward_auth` subrequest to `GET /api/checkauth/admin` for every request routed to them; the endpoint answers 200 only for a live administrator session and 401 otherwise. DBGate’s own basic-auth credentials are generated at deploy time and injected by Caddy into the proxied request, so the browser never sees them. The same admin-check endpoint doubles as the “last seen” heartbeat: every successful call updates the account’s last-login timestamp, which feeds the dashboard’s activity view.

5 The Test Lifecycle

5.1 One student, end to end

A student passes through the following stages (Figure 2):

1. **Registration.** `POST /api/student/register` with a desired nickname. The name is uppercased and stripped to A-Z 0-9 _; if free, the account is created with a random 8-digit access code, returned exactly once in the response. The frontend displays both with a “write this down” warning.
2. **Login.** `POST /api/login` with name and code sets the session cookie. The response body is either the literal string "OK" or a display-ready Lithuanian error message.
3. **Deal.** The first `GET /api/student/questions` triggers the deal: the test size is read from the `PhishingTestSize` setting (code-level fallback 30, admin-selectable values 9/12/15/21/30), a uniform random sample of that many *enabled* questions is drawn (clamped to the bank size), and the frozen snapshot described in Section 3.2 is bulk-inserted. Subsequent calls return the already-dealt set in the same order: the order is random per student and fixed at deal time, so the numbering in the navigation panel survives a reload. The pool is read once and the sample sized from that read, and a corrupt setting value falls back to the default like a missing one, so a first request cannot fail on a concurrent bank edit or a hand-edited setting.

4. **Answering.** The frontend posts the *entire* answer state to `POST /api/student/questions` after every click — verdicts (0 = real, 1 = phishing) into the answer rows, checkbox states into the option rows. Every update is scoped to the session's student ID, so a forged payload cannot write into another student's rows. The whole payload is validated before anything is written, so a malformed save is refused as a unit and never leaves a half-saved state; the frontend sends saves one at a time, coalescing clicks made while a save is in flight into a single follow-up, so an older state can never overwrite a newer one. Saving is idempotent and repeatable; a student can close the browser and resume with their name and code.
5. **Finish.** `GET /api/student/finish` takes the student's row lock, grades the frozen rows, writes the `TestResult` totals and sets the finished flag; a student who is already finished is left untouched, so a double click is harmless. The frontend waits for any pending autosave to be acknowledged before it navigates to this call, so the last click is always part of the frozen grade. The step is irreversible: from now on the questions endpoint returns an empty object for this student, and the route guard sends them to the results page.
6. **Results.** The student reads their own summary and per-question review — including the correct answers — through the same endpoints an administrator uses for any student; the public leaderboard shows their grade.

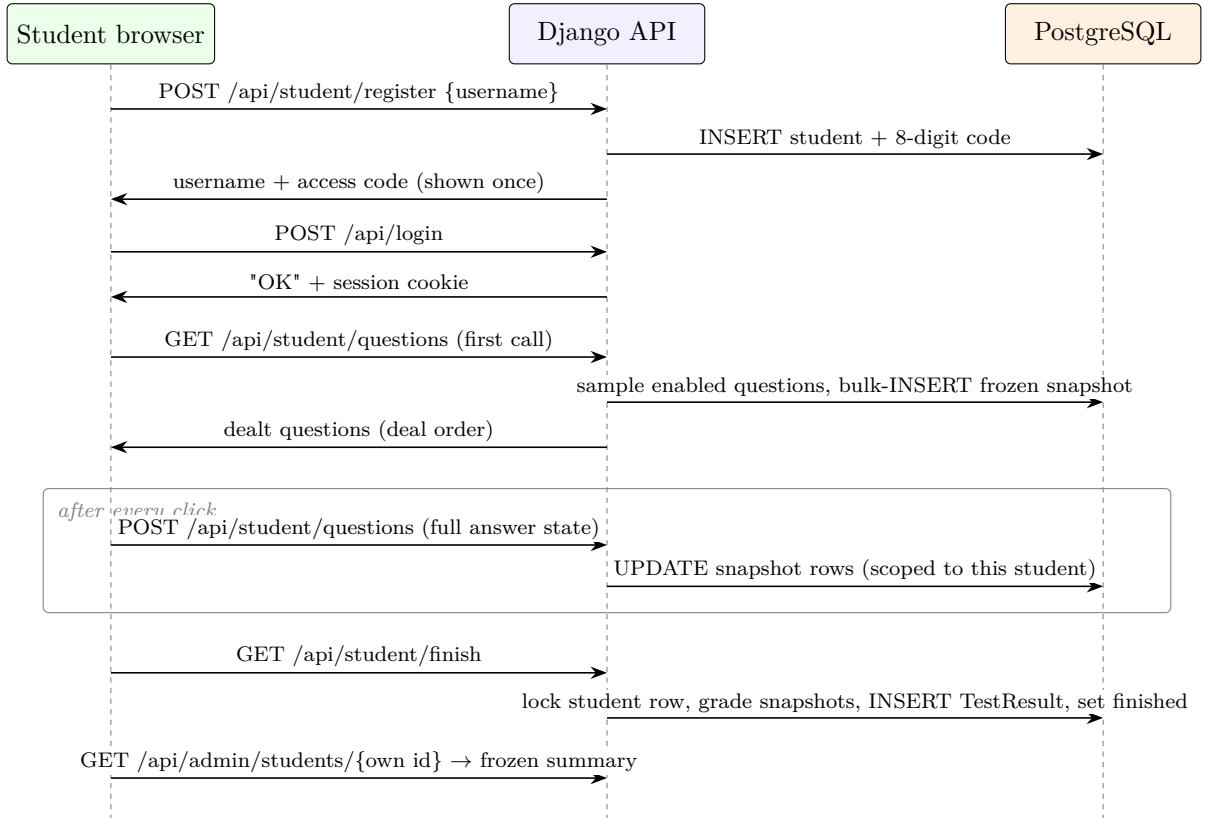


Figure 2: Message sequence for one student. The deal on the first questions request freezes the snapshot that all later answering and grading operates on; the finish call freezes the totals a second time, after which list views never re-judge this student.

5.2 Grading

Grading reads only the frozen snapshot. For a dealt question q let $v_q \in \{0, 1\}$ be the correct verdict, a_q the student's verdict, n_q the number of follow-up options, and c_q the number of options whose checkbox state matches the expected value (an untouched checkbox counts as unchecked). The question is worth at most one point, and the follow-up options act as a 0.1-point bonus/penalty band around it:

$$p_q = \begin{cases} 0 & \text{if } a_q \neq v_q, \\ \max(0, 1 - 0.1 n_q + 0.1 c_q) & \text{if } a_q = v_q. \end{cases}$$

That is, a correct verdict with every option right scores exactly 1; each missed option costs 0.1, down to a floor of zero (so a question with more than ten options can never subtract from the rest of the test). If the main verdict is wrong the question scores 0 regardless of the options. For a test of N dealt questions the final grade on the Lithuanian ten-point scale is

$$G = \text{round}\left(\frac{10}{N} \sum_{q=1}^N p_q, 2\right),$$

reported as a fixed two-decimal string (e.g. "8.50"). The summary additionally reports how many questions were identified correctly (verdict only), how many were *fully* correct (verdict and all options), the option totals, and the fully-correct share as a truncated integer percentage. The option totals are counted over the correctly identified questions only — exactly the options the formula scores — so a student who answered nothing shows 0/0 rather than credit for untouched checkboxes that happened to expect “unchecked”. N counts *dealt* questions, so unanswered questions count against the grade exactly like wrong verdicts. A newly added option expects “not checked” until the administrator says otherwise; an expected value that was never set at all (possible only on rows older than that rule) can never be matched.

For students still answering, these figures are computed live from the snapshot rows in two set-based queries; at finish they are frozen into `TestResult` and served from there permanently (Section 3.2).

6 The Web Interface

6.1 Technology

The frontend is a React 19 single-page application built with Vite 7, using MUI 7 for controls and data grids and Tailwind 4 for layout, with `react-router` 7 for client-side routing. There is no state management library and no client-side cache: one context provider checks `GET /api/checkauth` once per full page load and route guards redirect by role; server state is fetched by a small shared hook that supports interval polling and treats any 401 as a hard redirect to the login page. All user-facing text is Lithuanian (hardcoded — there is no i18n layer); the interface is styled in the faculty's burgundy (`#7B003F`) with a self-hosted Inter variable font whose latin-extended subset covers Lithuanian diacritics. The production image is built in two stages — a Node stage runs `vite build`, and the result is copied into a Caddy image — so the shipped container contains only static files and the SPA-fallback web server, no Node runtime.

A signature interaction pattern replaces confirmation dialogs throughout: destructive or irreversible actions (finish test, delete question, delete student, delete administrator) are *hold-to-*

confirm buttons that must be pressed for 1.5 seconds while a progress ring fills, on both mouse and touch.

6.2 The student experience

The login page defaults to *registration* — new students are the main audience. After registering, the username and access code are displayed once, in monospace chips under an amber “write this down” warning, with a single button that logs in and starts the test.

The test page shows one email screenshot at a time, rendered at half the viewport height with click-to-zoom into a fullscreen viewer (dismissed by backdrop click or **Escape**). Over the screenshot, the platform simulates the one behaviour a static image cannot provide: *link previews*. Invisible hotspot rectangles — defined by the administrator in fractional coordinates — track the rendered size of the image, and hovering one shows a black tooltip bubble with the link’s real URL just above the cursor, mimicking the status-bar preview of a real mail client. The hotspots are deliberately drawn fully transparent for students, so finding them requires actually inspecting the email.

Below the image sit the verdict buttons — *Tikras* (real) and *Fišingas* (phishing) — and the question’s follow-up checkboxes. Every click autosaves the entire answer state to the server (one save in flight at a time, later clicks coalesced) and a toast reports any failure, so losing a connection or closing the laptop costs nothing: logging in again resumes exactly where the student left off. A progress panel shows an answered counter, a progress bar, and a jump grid with one button per question (filled when answered, ringed when current). The finish button is a 1.5-second hold that waits for the last autosave to be confirmed before leaving the page — if it cannot be delivered, the student is told and stays on the test rather than finishing without those answers. The results page then shows the grade headline, three ratio tiles (fully correct, correctly identified, correct options), the student’s name and access code, and the same per-question review component the administrator sees — the answer key column in neutral grey, the student’s choices coloured green or red against it.

6.3 The administrator experience

Administrators get a collapsible sidebar shell (state persisted in `localStorage`) with the app pages and external links to the gated tools (slides filebrowser, dropbox filebrowser, Swagger, DBGate).

Dashboard. Polls the API every 2 seconds and shows the student count, the enabled/total question counts, and a live “currently solving” list — one progress bar per student active in the last 30 minutes, turning green at completion. The test size selector (9/12/15/21/30) sits directly in the questions widget and saves immediately on change; there is no separate settings page.

Question bank. A question is created by uploading a screenshot (drag-and-drop, image preview, 5MB limit); the new question then appears as a card to be filled in. Each card edits in place with a debounced autosave: 500 ms after the last keystroke the card saves and a status chip walks through *Saugoma.../Išsaugota* — there is no save button anywhere in the editor. Adding or deleting an option and deleting a question save immediately (they create or remove rows); deletions are hold-to-confirm. Each card carries the phishing verdict checkbox, the optional extra-instruction text, the option list with per-option correct values, and an enable switch — only enabled questions are dealt, and a banner warns when fewer questions are enabled than the configured test size. A filter bar searches by text, option text or ID; filtered-out cards are hidden rather than unmounted so unsaved edits survive filter changes.

The hotspot editor opens fullscreen over the card: link areas are drawn, dragged and resized directly on the screenshot (`react-rnd`), each with its URL, and saved back as fractional coordinates. The same editor is reachable directly at `/admin/questions/{id}`.

Students. A data grid (5-second polling, quick filter, default “last month” switch) lists every student with progress, grade and timestamps, sorted as numbers and dates rather than text. The detail page shows the credential card — including the access code, so a lecturer can rescue a student who lost theirs — the summary tiles, and two tabs: the per-question review and a condensed table with screenshot thumbnails. Students can be deleted (hold-to-confirm); the cascade removes their dealt test and grades but never touches images.

Administrators. A grid plus a single modal for create/edit/delete against one endpoint; an empty `id` signals insert and an empty password on edit means “keep the current one”. Passwords must be 8 characters to 72 bytes, email addresses unique, and an administrator can neither disable nor delete their own account — so at least one working administrator always survives.

6.4 Public displays

Two unauthenticated pages are designed for a projector:

- **Leaderboard** (`/leaderboard`) — ranks students by grade with gold/silver/bronze badges, shows blue progress bars for students still answering and burgundy grade bars for finished ones, refreshes every 5 seconds with a visible countdown, and by default filters to students seen in the last 24 hours.
- **Slides kiosk** (`/slides`) — alternates between the leaderboard (embedded in an iframe behind a click-blocking overlay, 6 seconds) and a random slide image served from the slides directory (20 seconds). The next slide is fetched and decoded as a blob *while the leaderboard is showing*, so the swap is always instant; administrators manage the slide files through the slides filebrowser.

7 Deployment and Operations

7.1 Prerequisites

A Linux host with Docker and Docker Compose, and port 80 free. There are no other dependencies: images are pulled from Docker Hub (`vuknf/fisingas-*`), and each service also carries a commented-out `build:` variant in the compose file for building locally.

7.2 Starting the stack

The only manual step is creating the compose file from the sample; everything else is one idempotent script:

```
cp docker-compose.yml.sample docker-compose.yml
./runUpdateThisStack.sh
```

The script creates the `_DATA` directories (`postgres`, `slides`, `dropbox`) and fixes their ownership, checks that Docker is installed, generates `DJANGO_SECRET_KEY` and the DBGate credentials into `.env` (once), creates the Docker network, (re)starts the stack, applies database migrations in a retry loop (the first PostgreSQL start takes a few seconds), and — only on an empty database — creates the first administrator account and prints its credentials:

```
Email:    admin@admin.com
Password: admin
```

This password must be changed immediately on the Administrators page. The service is then reachable on port 80 of the host: `/` is the application, `/swagger` the API documentation, `/dbgate` the database UI (all three tool paths admin-gated).

Every request is logged by Gunicorn to the container's standard output (`docker logs fisingas-django`): the client address as forwarded by Caddy, the request line, status, response size and duration.

7.3 Data retention

The cron container — a BusyBox `crond` with the Docker CLI, no network, and the crontab baked into the image — runs one job: every night at 05:00 (an hour that exists on every day of the year; 03:00 is the one daylight-saving time skips in spring and repeats in autumn) it executes the `delete_old_students` management command inside the Django container. The command deletes student accounts whose last login *and* registration are both older than 180 days; the cascade removes their dealt tests and grades, while question images are never touched. Fresh accounts that never logged in survive. Everything else is kept indefinitely: the question bank, the images, the administrator accounts, and the `_DATA` directories on the host.

7.4 Configuration reference

All settings are environment variables set in `docker-compose.yml` or generated into `.env` (Table 3). The single application-level setting — the test size — lives in the database and is edited from the dashboard.

Variable	Default	Meaning
DJANGO_SECRET_KEY	generated into <code>.env</code>	Signs session default
DATABASE_URL	<code>postgres://fisingas:...@fisingas-postgres:5432/fisingas</code>	PostgreSQL
SLIDES_DIRECTORY	<code>/slides</code>	Slide images
DJANGO_DEBUG	unset (False)	Switches the server with d
POSTGRES_DB / _USER / _PASSWORD	<code>fisingas</code>	Database bo
DBGATE_PASSWORD	generated into <code>.env</code>	DBGate basi
DBGATE_AUTH_HEADER	generated into <code>.env</code>	Pre-encoded jects toward

Table 3: Configuration reference. The `PhishingTestSize` setting (questions per test; code fallback 30, dashboard choices 9–30) is stored in the database, not the environment.

7.5 Development mode

Every service block in the compose file carries commented `Dev` lines: switching them on bind-mounts the source into the containers and starts the Vite dev server (with hot reload) and Django's `runserver` (via `DJANGO_DEBUG=True`) instead of the production servers. The backend image pins its five dependencies directly in the Dockerfile — there is no `requirements.txt` — and companion `pushDockerhub.sh` scripts build and push each image with `latest` and date tags.

7.6 Regression tests

`django/runTests.sh` builds the backend image and runs the regression suite — over three hundred tests — in a throwaway container against an in-memory SQLite database, so no running stack is needed. The suite covers every endpoint, the grading arithmetic, the snapshot and freeze discipline, the retention sweep and the timestamp policy; a contract layer validates live responses against `swagger/swagger.yaml`, so the API documentation cannot drift from the code unnoticed. Known, not-yet-fixed defects are pinned as *expected failures*: each has a test asserting the correct behaviour, so the suite stays green while the defect stays documented and executable, and a fix flips the test instead of passing silently.

8 Limitations and Future Work

The source-level view also makes the system’s boundaries clear. The platform is built for a supervised classroom on a trusted network, and several choices only make sense in that setting:

- **Plain HTTP.** The endpoint listens on port 80 with no TLS; production deployments should terminate TLS in front of the endpoint. The session cookie carries no `Secure` flag, so transport security is entirely the perimeter’s job.
- **No rate limiting.** Login and registration are unauthenticated and unthrottled. Student access codes are 8-digit numbers, so an online brute-force is a numbers game the platform itself does nothing to stop; likewise nothing bounds mass registration. Access codes and the question sample are drawn from Python’s default PRNG, not a cryptographic one.
- **CSRF protection is the Origin check only.** Django’s CSRF middleware is off by design; Caddy rejects foreign origins, but requests without an `Origin` header pass, and the finish endpoint is a state-changing GET — a crafted cross-site navigation could irreversibly finish a logged-in student’s test.
- **The answer key is available to a determined student mid-test.** The self-access rule on the results endpoints has no finished-check, so a student who reads the API (or the Swagger docs, were they not admin-gated) can fetch their own graded answers — including correct verdicts — while still answering. Similarly, any authenticated student can fetch any question image and its link areas by ID, not just their dealt ones. Honest use is unaffected; unsupervised high-stakes use would need a finished-gate on these endpoints.
- **The leaderboard is public by design** and discloses usernames, grades and last-seen times to anyone who can reach the host. Students choose their own nicknames, which mitigates but does not remove the disclosure.
- **No audit trail.** Administrator actions and logins are not recorded beyond Unicorn’s request log, and there is no undo for a deleted student or question. Images live as database blobs, which is simple and consistent but makes the database the size bottleneck.
- **No time limit and no start timestamp.** Nothing records when a test was dealt, so the duration of a sitting can neither be enforced nor reconstructed afterwards; only the finish time is stored.
- **Single language.** All interface text is hardcoded Lithuanian; internationalisation would require an i18n layer in the frontend.

9 Conclusion

Fisingas shows that a purposeful classroom assessment platform does not require heavyweight infrastructure. One Caddy ingress, a thin Django API, PostgreSQL, and a React SPA — nine containers, one script — deliver the full loop: self-service registration with generated access codes, a visual test whose simulated link previews teach the single most valuable phishing-detection habit, live progress on the lecturer’s dashboard, and a projected leaderboard that turns the exercise into a game. The defining engineering decision is the snapshot discipline: dealing freezes question content into the student’s rows and finishing freezes the totals, so the question bank can evolve freely — mid-course, even mid-test — without ever disturbing a grade already given. The entire stack is reproducible from one compose file and operable by anyone comfortable with Docker.